Introduction

The purpose of this project is to explore different structures within the **Maps ADT**, and combine them to form complex data stores in application. There are three objectives:

- **Implement:** Apply what you learned in class about Maps to implement a Linear Probing HashMap.
- **Refactor:** Refactor an AVL into a TreeMap to add useful functionality.
- **Apply:** Solve database challenges using the Maps you've created.

In this project, you will implement a HashMap as taught in lecture. Then, you will complete and refactor an AVL to create a TreeMap, which is another implementation of the Map ADT. Finally, you will implement methods to create a simple database using your TreeMap and HashMap.

This is a complex assignment, so start early and ask many questions! Piazza is a great resource for viewing other students' questions and getting quick answers from TAs. Feel free to visit Office Hours for more direct help.

If you complete this project **by yourself**, you will learn a lot and build your competency in solving difficult problems with code, which will be extremely useful for future coursework, passion projects, and job interviews. Thus, we require that you do not use AI to ideate, debug, or write code, nor can you copy code from other students. Abusing AI will hurt your grade in the Demo and Exam. **AI usage of any kind is forbidden on this project. Violators will receive a zero and be referred to OSI.**

Time and space complexity is important for this assignment. Solutions that do not fit within the stated requirements will receive significant point penalties.

Provided Files

`Main.java`	The main class for your project. You will provide instantiation of your created instances, which is necessary to test your code.
`HashMap.java`	Shell class for you to complete to implement a HashMap.
`MapEntry.java`	A key-value pair object class for use in the HashMap.
`AVL.java`	Partially-completed AVL class for you to complete.
`AVLNode.java`	An AVL node object class for use in the AVL.
`StaticTreeMap.java`	An interface for you to extend and implement a TreeMap with.
`StaticDaleDB.java`	An interface for you to extend and implement for your database.
`DaleRecord.java`	An interface that represents and contains records in our DaleDB database. You will use this to access record types. For more information on the `record` keyword, see Oracle's documentation.
`HashMapStudentTests.java`	Part 1 local testing file (non-comprehensive).
`AVLStudentTests.java`	Part 2 local testing file (non-comprehensive).
`TreeMapStudentTests.java`	Part 2 local testing file (non-comprehensive).
`DaleDBStudentTests.java`	Part 3 local testing file (non-comprehensive).

Deliverables

You will need to create your own files that implement the interfaces we provide. It does not matter what you name the files as long as you follow the directions for inputs/outputs in Main.java.

You must submit **all** files that you create, along with your modified Main.java and HashMap.java. To submit your code, follow the process below depending on your operating system:

Windows: With your project open in File Explorer, double click on `submit-windows.bat`

MacOS/Unix: With your project open in your Intellij, open the integrated terminal (the [>_] icon). Run the following command: `./submit-unix.sh`. If the file isn't executable, try `chmod +x submit-unix.sh`.

These will create a file called `submission.zip` inside your project folder, and this is what you should upload to Gradescope.

Once submitted, double-check that it has uploaded properly on Gradescope. It will tell you if your code failed to compile.

Getting Started

1. **Download the files**: navigate to the Canvas assignment, and download and unzip the folder.
2. **Setup the project**: if you are using IntelliJ, open the project in the root directory of the unzipped project files. The 'src' directory should already be marked as sources root (blue) the 'tst' directory should already be maked as test sources root (green). *Optional tip: right-click on the icon next to the top line of any Javadoc and select 'Render All Doc Comments' to view the Javadocs in a user-friendly format.*
3. **Complete parts 1 and 2 first**: these parts are hard prerequisites to part 3. Any classes you create in these parts must be located in their respective packages.
4. **Complete part 3**: use the data structures from 1 and 2 to solve the scenario in part 3. Any classes you create in this part must be located in the respective package.
5. **Test locally**: ensure you fill out the `Main.java` file with your instances from parts 2 and 3. From there, you can run the student test files located in `tst`. Note that these files are not comprehensive, and you are encouraged to write your own JUnits to test edge cases.
6. **Submit to Gradescope**: follow the above section's steps to zip your files and upload that to Gradescope, which will run the full suite of tests.

Part 1: HashMap

Background

A Map is an abstract data type (ADT) that associates a key with a value. Maps are fundamental to computer science. Maps are such an important data structure, in fact, that we have dedicated an entire project to them!

A HashMap uses "hashing" to map unique keys to values, and allows $O(1)$ average-case lookup of a value when the key is known. We omit most implementation details of the HashMap from this document since you learned it in lecture. Take a look at the Module videos if you need a refresher.

As a reminder, the HashMap should **not** contain duplicate keys, but **can** contain duplicate values. In the event of trying to add a duplicate key, replace the value in the existing (key, value) pair with the new value and return the old value.

There are several ways to implement a HashMap. The two types we learn in CS 1332 are Probing (open-addressing) and Chaining (closed-addressing). In Part 1, you will implement a **Linear Probing HashMap**.

Probing

Probing is a strategy for handling collisions. Recall that a **collision** is when we attempt to insert data at a location that already contains data. For example, if we put "3" at index 3 of the array, and then our hash function tells us to insert "30" at index 3 too, we encounter a collision.

With probing as our collision resolution strategy, we iteratively try new locations until we find a spot to insert the data. **Linear** probing will search linearly (one spot at a time) for a spot, whereas **Quadratic** probing will search quadratically (one spot away, then two away, then four away, then nine away, and so on). **For this project, you should utilize a Linear probing collision resolution strategy.**

Put

The `put` method is the most complicated method in the HashMap, as it does most of the work to ensure the data is organized for efficient access. There are several edge cases to keep track of. Here is a list of some cases to consider:

- Adding an entry with a brand new key (insert)
- Adding an entry with a key that is already present in the map (update)
- Adding this entry would exceed the load factor (resize)
- Hashing to an index that is already occupied (collision)
  - Probing and encountering a null
  - Probing and encountering a DEL flag
  - Probing and encountering a DEL flag *with the same key as the new entry*

Make sure that you implement each of these cases, and each method in the HashMap, as efficiently as possible. You must call `resizeBackingTable(length)` when you need to resize.

Load Factor and Resizing

A `HashMap` will lose efficiency if it becomes too full. To combat this, if adding to the table would cause the load factor to **exceed** (greater than, not greater than or equal to) the `MAX_LOAD_FACTOR` constant, the table should be resized to have a capacity of $2n + 1$, where n is the current capacity before adding the new entry.

To resize, you must **re-hash** all of the entries into the new table. You cannot simply copy to the same index – think about why. You should not copy over deleted entries.

Remove and DEL Flags

Since linear probing is an open addressing scheme, you should not set removed entries to null. Instead, you need to implement a "soft remove" using the removed flag in `MapEntry.java`. The flag is a property of each entry, and it keeps track of if the entry has been removed.

From the user's perspective, removed entries no longer exist in the map, but under the hood we keep them to help with probing. When removing, do *not* null out the entry; instead, change the flag for that entry. This allows us to probe to all entries in other methods without stopping prematurely – again, think about why.

Iterator

In addition to all of the functional methods, you will also have to implement `Iterable`. This means that you will need to create and return an `Iterator` that allows you to iterate through the HashMap (i.e., through a while or for-each loop).

The iterator should return the keys from left to right in the backing array.

The iterator must use $O(1)$ auxiliary space. You cannot call `keySet()` and iterate through that, as that is $O(n)$. Your `next()` method should throw a `NoSuchElementException` if there is no next element. You do not need to worry about concurrent modification.

In the HashMap below, an iterator should first return **0**, then **11**, then **6**, and finally **17**, ignoring empty (null) indices and deleted entries.

0	1	2	3	4	5	6	7	8
<0,A>		<11,R>	<21,G> DEL			<6,F>	<7,G> DEL	<17,L>

Iterator output: 0, 11, 6, 17

Local Testing

To test your HashMap locally, you can run the `HashMapStudentTests.java` file, which contains several JUnits to test basic functionality. These tests are **not comprehensive**, meaning they do not cover all edge cases. You should write more tests yourself if you want to verify that edge cases are handled.

Part 2: TreeMap

Background

Now that you know how to implement a working HashMap, you can extend this to a more complex data structure belonging to the Map ADT: a TreeMap!

A TreeMap is just what you might expect: a Map implemented by a tree backing structure, rather than an array. For this project, you will use an **AVL** backing structure to implement a TreeMap.

Unlike a HashMap, a TreeMap has a deterministic iteration order; that is, it is **key-ordered**. This should make sense, since all of our BST-type structures have an order property we can take advantage of. This property will be very useful in the application section of this project.

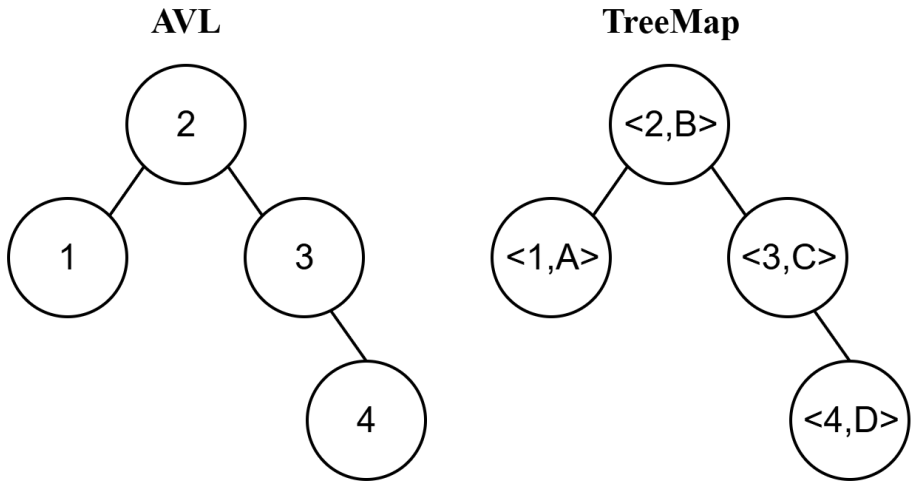
Note: If you try to search "TreeMap" on the internet, you may run into an unrelated data visualization of the same name. You can learn about this later in CS 4460 (InfoVis), but for now, don't get confused by the shared name.

AVL vs TreeMap

An AVL is a self-balancing tree structure with nodes that each store a **single value**. A TreeMap is a self-balancing tree structure with nodes that each contain a **key-value pair**, and these entries are stored in a deterministic order based on the key. See AVL and TreeMap on CSVistool if you are confused.

Part 2 involves creating both an AVL class and a TreeMap class. **We will not directly test your AVL code**; instead, you should write the AVL class first and test it locally to ensure your logic is correct *before* tackling the TreeMap.

Then, you will re-use your AVL logic to create a TreeMap class. You can implement TreeMap however you like (as long as it does not miss any requirements in the Javadocs or this PDF), but you will likely want to create an independent class that does not inherit from or contain an AVL. Feel free to copy-and-paste code where applicable.



Adelson-Velsky Landis (AVL) Tree

To begin, you need to complete an AVL implementation. **We provided a mostly-complete AVL file for you to refactor**. It is very similar to your Homework 2, but with additional logic for self-balancing.

First, you should read through all of the existing code in `AVL.java` so that you understand what you have to work with. Then, implement these four methods such that they make the AVL self-balance with rotations, as taught in lecture:

- **update(node)** → update a node's height and balance factor attributes
- **leftRotate(node)** → perform a left rotation on a node
- **rightRotate(node)** → perform a right rotation on a node
- **balance(node)** → check if a node needs to be balanced, and then balance the subtree

Note that for `remove`, this AVL uses the **predecessor** for the two-child case. Your TreeMap should also use the predecessor where applicable.

Test your work by running the `AVLStudentTests.java` file. Solve any issues before attempting TreeMap.

Creating a TreeMap

To think about how to approach implementing the TreeMap, consider your existing AVL code. **How can you refactor the code to support the Map ADT methods?** The methods you will need to implement are in `StaticTreeMap.java`:

- **put(key, value)** → puts a key-value pair and returns its previous value if one is replaced
- **remove(key)** → removes a key-value pair and returns the removed value
- **get(key)** → returns the value associated with the key
- **getRange(lower, upper)** → returns an in-order list of values for keys between two bounds
- **containsKey(key)** → returns true if the key is in the map
- **keySet()** → returns a set of all keys
- **values()** → returns a list of all values in sorted order by key
- **clear()** → empties the map and returns nothing
- **height()** → returns the height of the tree backing the map
- **size()** → returns the size of the map
- **iterator()** → returns an iterator that can iterate through the **values** of the TreeMap in-order

To begin Part 2, create a new Java class that *implements* the StaticTreeMap interface. Once you have made your new class, write out all of the method headers from the interface so that the compiler is satisfied. You can do this easily using CTRL+I or CMD+I in IntelliJ. Then, begin implementing each method!

This is where you will want to reference your working AVL. The `put` method will use logic from your AVL `add`, the `get` and `remove` methods will similarly relate to each other, and so on.

Use CSVistool.com to visualize how your TreeMap should behave.

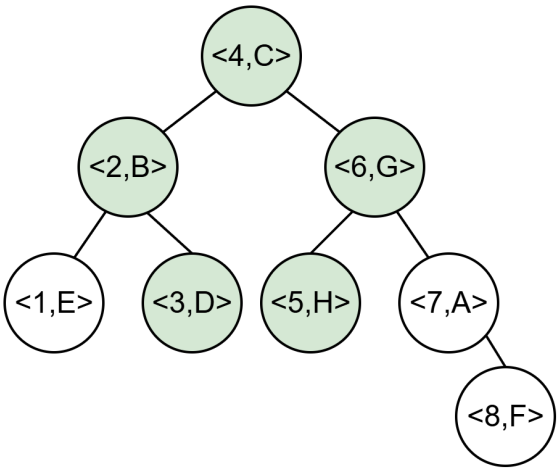
GetRange

A common use case for TreeMaps is to find all of the records within some range of keys. Because the entries are stored in a predictable order in the TreeMap based on the keys, we can efficiently find and return this range.

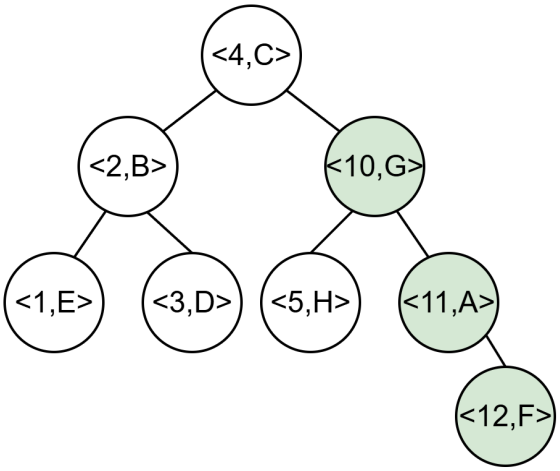
A lower and upper bound are passed into the method. You should return the list of all values, in order, whose keys are between `lower` and `upper` (inclusive).

Your code must have a time complexity of $O(log(n) + k)$, where k is the number of elements within the range and is practically less than n. The space complexity must be $O(1)$ excluding the recursive stack and output; in other words, you cannot store any data except for the list of values you are returning.

You should throw an `IllegalArgumentException` if either bound is null. If there are no values within the range (or the range is impossible), return an empty list. The bounds are not necessarily keys that actually exist in the map.



Example 1: On the TreeMap above, `getRange(2,6)` should return [**B,D,C,H,G**] in this exact order.



Example 2: To see how the bounds do not need to be in the TreeMap, `getRange(7,15)` should return [**G,A,F**] in this exact order.

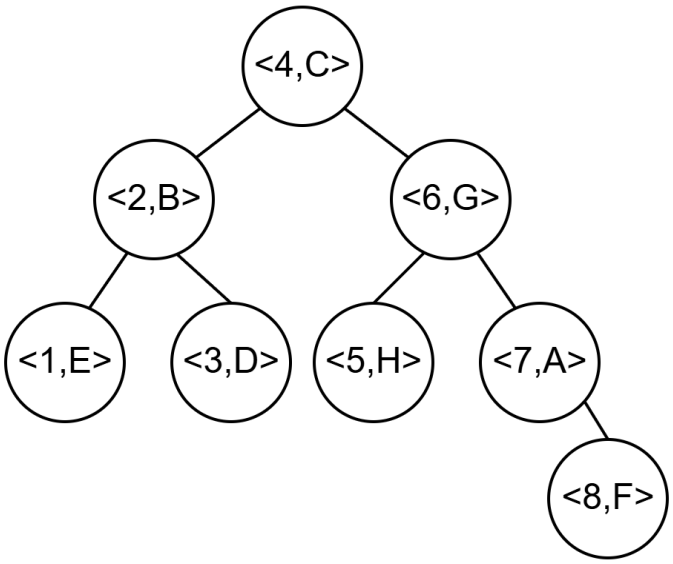
Iterator

In order to satisfy the Iterable requirement of the TreeMap, you will need to implement a method returning an iterator. It is highly recommended that you create a **private inner class** and return an instance of that class.

The iterator should allow users to iterate through the data in a *deterministic* order; that is, the order is the same every time. You may use additional data structures as long as they use at most $O(\log n)$ auxiliary space, so you cannot simply call or rewrite `values()`. Each `next()` call should have a worst case runtime of $O(\log n)$, but on average should be $O(1)$.

The iterator must return the values ***in-order by key*** from least to greatest. That is, the smallest key's value should be the first item returned by the iterator, and the largest key's value should be the last item. *Hint: You will want to avoid using recursion for this iterator, as the recursive call stack does not persist across iterator calls.*

In the example below, the calling `next()` should return **E**, then **B**, then **D**, and so on, with the last value produced by the iterator being **F**.



Iterator output: E, B, D, C, H, G, A, F

Local Testing

To test your TreeMap locally, you can run the `TreeMapStudentTests.java` file, which contains several JUnits to test basic functionality. These tests are **not comprehensive**, meaning they do not cover all edge cases. You should write more tests yourself if you want to verify that edge cases are handled. You will need to complete the relevant method in `Main.java` before testing.

Note that for all methods in this part, the tests count comparisons as a way of checking efficiency. Carefully consider how many nodes you visit and how many comparisons you make per node in your recursive methods. Also note that, in order to test the TreeMap structure, all tests rely on basic functionality of `put` `get`, `keySet` and `values`. Ensure these methods are completed prior to submission on Gradescope.

Part 3: DaleDB

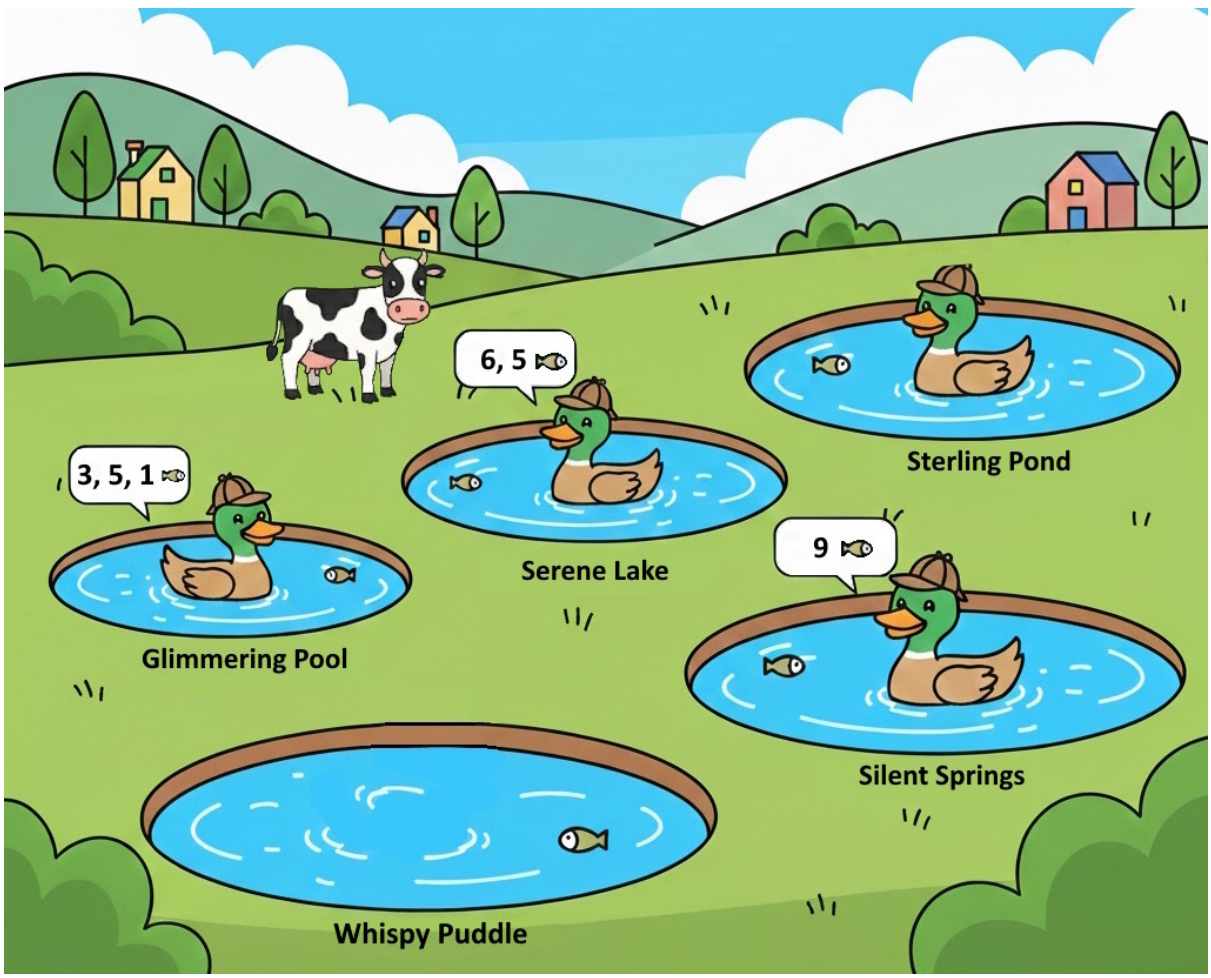
Note: You cannot use any Java-provided Map structures in this project – you may only use the Maps you created.

Background

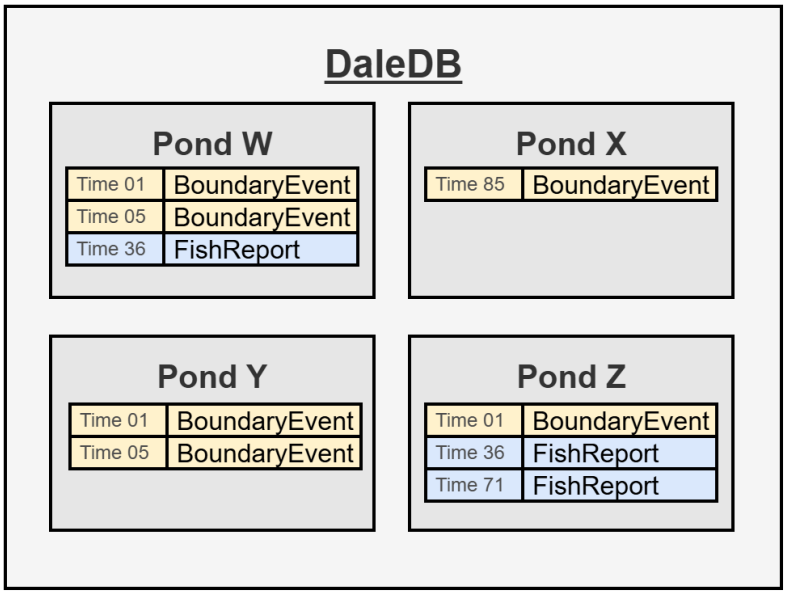
The CS 1332 Duck Registry is requesting your assistance to build and maintain the DaleDB, named after the first known duck, Dale. The Registry's goal is to track activity across many ponds at scale.

Our world is simple: there are **Ponds**, **Ducks**, **Fish**, and **Cows**.

- Ducks fly around, and sometimes enter and exit ponds. These movements are tracked in the database.
- When at a pond, a duck can go fishing. Every so often, a duck will report its caught fish to the database.
- Sometimes, cows wander into or out of a pond area. Their name is recorded alongside the duck names, but it is unknown which names are cows and which are ducks.
- To save space in the database, we prioritize recording information for active ponds. If a pond has little or no activity, its records may be destroyed (see evict).



Understanding DaleDB



With the example above, we can see the high-level structure of DaleDB. An instance of DaleDB is a container that holds any number ponds, where the database is effectively split up across different sections (called **database partitions** in real life). In this case, the partitions are the ponds.

Ponds are identified by a unique name string, which *can* be an empty string. Each pond itself contains any number of records. There are two `DaleRecord` types:

- **BoundaryEvent**: marks entrances and exits at pond boundaries, listing the animals involved and the number of cows detected among those listed animals.
- **FishReport**: stores a duck's list of fish weights caught during a fishing event.

These are records (subclasses but which only store data, not methods) within the DaleRecord interface. Read through the `DaleRecord` interface to learn more about the structure of the data in each case.

Every record has an associated **timestamp**, which is unique within the pond. This means that the timestamp effectively serves as a unique key for each record. In any single pond, there are no two records with the same timestamp.

Together, a combination of a pond and timestamp is guaranteed to be unique; that is, there will not be any records with the same pond and timestamp in combination. New records arrive *out of order*. The database must meet the following general rules:

- An arbitrary record lookup time of $O(\log r)$, where r is the number of records in a pond.
- Efficient, ordered scanning of a pond in $O(r)$ time.
- Identify inactive ponds to be evicted in $O(1)$ time.

Specific requirements for each method are listed in the following pages.

Part 3a: Basic Methods

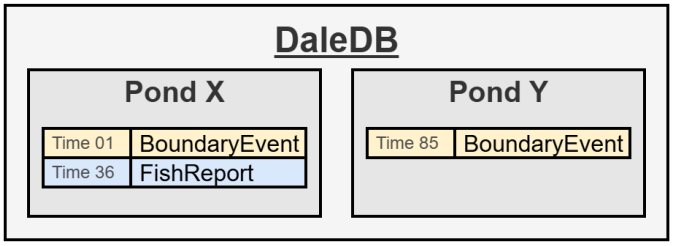
1. putRecord

This method accepts any type of record and inserts it into the database, or modifies it if it already exists, both in $O(log(r))$ time. Return the old `DaleRecord` if it was modified, otherwise return `null`.

If the provided record is null, throw an `IllegalArgumentException`. You can assume that a non-null record will be valid (i.e., it will not contain any null data), so you need not validate the records. Note that the record's data includes the pond name.

Example: After the following (simplified) operations, DaleDB should conceptually look like this.

- `putRecord(new BoundaryEvent(pond="X", timestamp=1, type=ENTRANCE, ...))`
- `putRecord(new FishReport(pond="X", timestamp=36, weights=[0.4, 0.5, 1.2], ...))`
- `putRecord(new BoundaryEvent(pond="Y", timestamp=85, type=ENTRANCE), ...)`



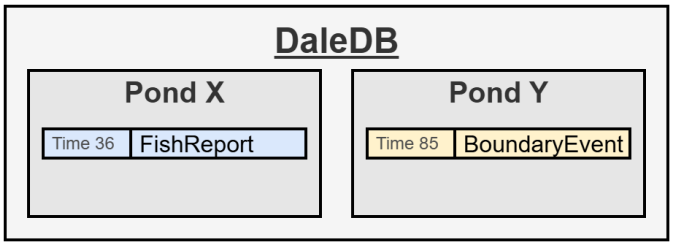
2. deleteRecord

This method accepts a pond name and timestamp, and deletes the record with that timestamp from that pond in $O(log(r))$ time.

Return the deleted `DaleRecord`. If the pond name is null, throw an `IllegalArgumentException`. If the record does not exist, throw a `NoSuchElementException`. Note that for all methods, an empty string is a valid pond name, but a null string is an invalid input.

If a pond has zero records as a result of this operation, the pond must be removed from the database.

Example: Carrying on from the previous example, after calling `deleteRecord("X", 1)`, DaleDB should conceptually look like this.

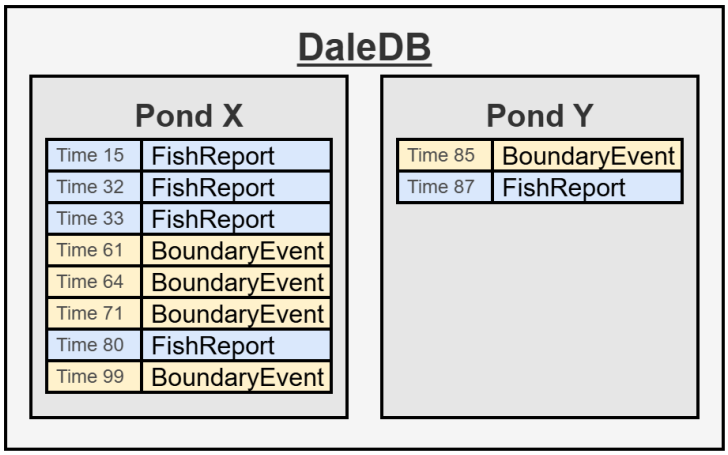


3. getRecord

This method accepts a pond name and timestamp, and returns the record with that timestamp from that pond in $O(log(r))$ time.

If the pond name is null, throw an `IllegalArgumentException`. If the record does not exist, throw a `NoSuchElementException`.

Example: When calling `getRecord("Y", 85)` in the below example, DaleDB should return the BoundaryEvent record at time 85 from Pond Y.



4. getPond

This method accepts a pond name and returns a list of all records in that pond in $O(r)$ time. The list must be sorted in ascending order by timestamp.

If the pond name is null, throw an `IllegalArgumentException`. Note that an existing pond should never be empty, as empty ponds are removed from the database. If the pond does not exist, you should return an empty list as there are no records associated with this pond name.

Example: When calling `getPond("X")` in the previous example, DaleDB should return the list of every record in Pond X (some data is omitted for simplicity):

```
output = [
    FishReport(pond="X", timestamp=15, ...),
    FishReport(pond="X", timestamp=32, ...),
    FishReport(pond="X", timestamp=33, ...),
    BoundaryEvent(pond="X", timestamp=61, ...),
    BoundaryEvent(pond="X", timestamp=64, ...),
    BoundaryEvent(pond="X", timestamp=71, ...),
    FishReport(pond="X", timestamp=80, ...),
    BoundaryEvent(pond="X", timestamp=99, ...)
]
```

Note that the records are listed in order of timestamp, and the Pond Y records are not included.

5. getPondCount

This method simply returns the number of ponds in the database in $O(1)$ time. In the example below on this page, there are two ponds, Pond X and Pond Y, so the method call would return 2.

6. getRecordRange

This method accepts a pond name, start timestamp, and end timestamp, and returns a list of all records in that pond in the inclusive range [start,end]. The returned list must be sorted in ascending order by timestamp.

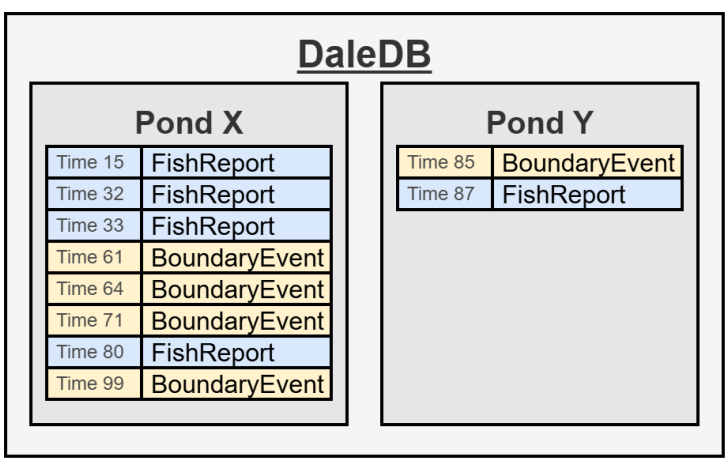
If the pond name is null, throw an `IllegalArgumentException`. Note that an existing pond should never be empty, as empty ponds are removed from the database. If the pond does not exist, throw a `NoSuchElementException`.

If there are no values within the range (or the range is impossible), return an empty list. The bounds are not necessarily timestamps that actually correspond to a record.

Your code must have an $O(r)$ time complexity and $O(log(r))$ auxiliary space complexity (which excludes the output list).

Example: When calling `getRecordRange("X",33,86)` in the example below, DaleDB should return the list of every record in the range [33,86] in Pond X (some data is omitted for simplicity):

```
output = [
    FishReport(pond="X", timestamp=33, ...),
    BoundaryEvent(pond="X", timestamp=61, ...),
    BoundaryEvent(pond="X", timestamp=64, ...),
    BoundaryEvent(pond="X", timestamp=71, ...),
    FishReport(pond="X", timestamp=80, ...)
]
```



Part 3b: Challenge Methods

7. evict

Given an integer k, remove the k ponds that have been accessed the least recently. Return the records that were removed as a result.

NOTE: The order ponds are accessed in is unrelated to the records' timestamps. A pond is accessed when a user calls some method specific to that pond, like `putRecord` or `getRecord`.

The removed records should be returned as a list, sorted first by ascending order of eviction, and second by ascending order of timestamp. Throw an `IllegalArgumentException` if k is zero or negative, and a `NoSuchElementException` if there are less than k ponds in the database.

Deciding which pond(s) to evict should be $O(1)$. Technically, this method has a time complexity of $O(kr)$. Note that p, the number of ponds, is not included, so it is constant time with respect to the ponds.

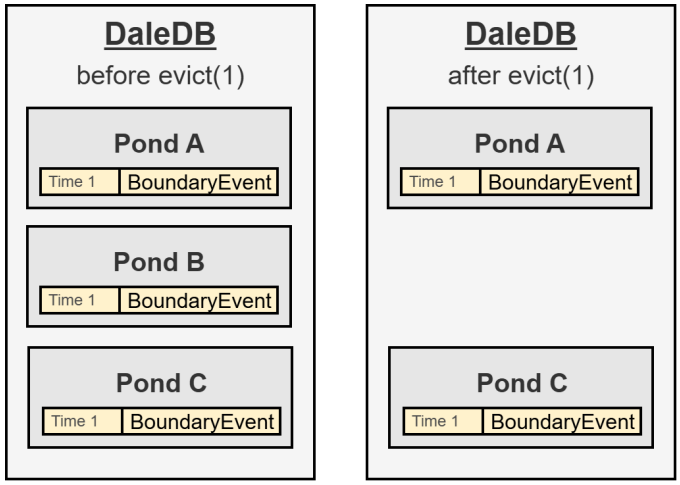
Note: implementing the evict functionality will impact many of your other methods in DaleDB, as many of the DaleDB operations are pond-specific.

Example 1: Say we perform the following operations in order:

1. `putRecord(new BoundaryEvent(pond="A", timestamp=1, ...))`
2. `putRecord(new BoundaryEvent(pond="B", timestamp=1, ...))`
3. `putRecord(new BoundaryEvent(pond="C", timestamp=1, ...))`
4. `getPond("A")` $\rightarrow$ ***The access order is now A $\leftarrow$ C $\leftarrow$ B***
5. `evict(1)` $\rightarrow$ **returns** `[BoundaryEvent(pond="B", ...)]`

Pond B's record is returned, and Pond B is removed from the database, because **Pond B was the least-recently-accessed pond**. Remember, each time a pond is read or written to by an operation, it becomes the most-recently-accessed. The first pond to be evicted will be the one which was *least* recently accessed.

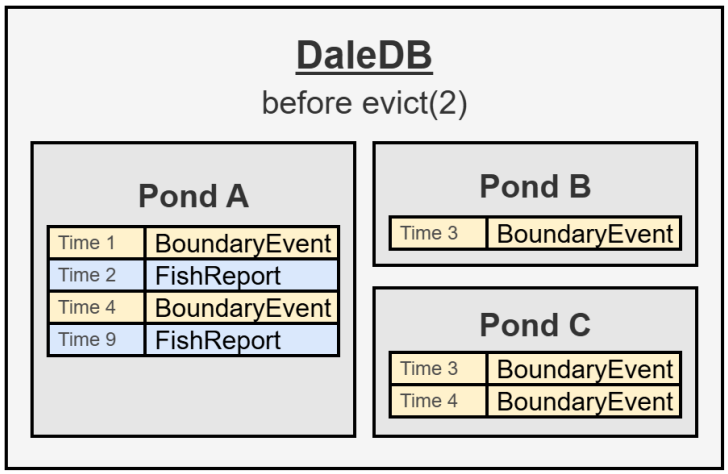
Since we called `getPond` on Pond A, it is now the most recently accessed pond, and as a result, the new least recently accessed pond is B. Thus, Pond B is evicted.



Example 2: Say we perform the following operations in order:

Operation	**Current Access Order**
`putRecord(new BoundaryEvent(pond="B", timestamp=3, ...))`	B
`putRecord(new FishReport(pond="A", timestamp=9, ...))`	A ← B
`putRecord(new FishReport(pond="A", timestamp=2, ...))`	A ← B
`putRecord(new BoundaryEvent(pond="A", timestamp=4, ...))`	A ← B
`putRecord(new BoundaryEvent(pond="C", timestamp=4, ...))`	C ← A ← B
`putRecord(new BoundaryEvent(pond="A", timestamp=1, ...))`	A ← C ← B
`putRecord(new BoundaryEvent(pond="C", timestamp=3, ...))`	C ← A ← B

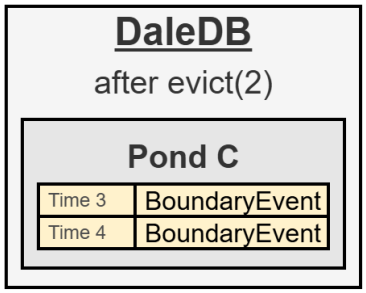
The state of the database is now:



With the access order (from most to least-recently accessed) of **C, A, B**.
Calling `evict(2)` will delete Ponds B and A, and return the following output in order:

```
output = [
    BoundaryEvent(pond="B", timestamp=3, ...),
    BoundaryEvent(pond="A", timestamp=1, ...),
    FishReport(pond="A", timestamp=2, ...),
    BoundaryEvent(pond="A", timestamp=4, ...),
    FishReport(pond="A", timestamp=9, ...)
]
```

And the database now only contains Pond C:



8. getPeakConcurrentOccupancy

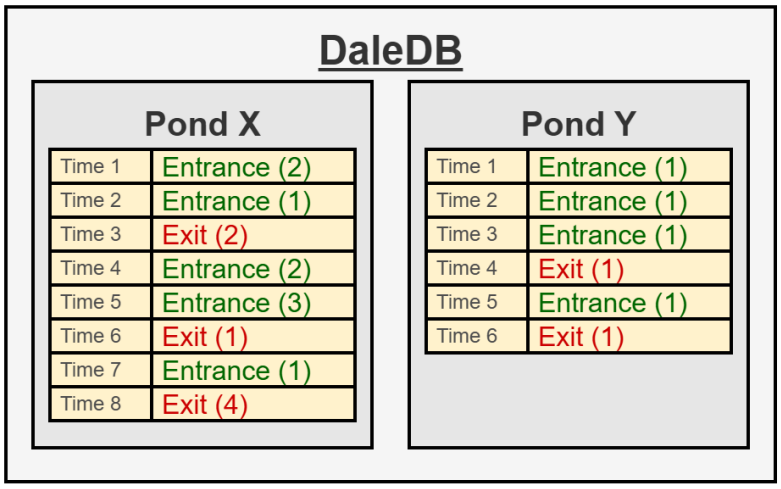
This method accepts a pond name, and returns the maximum number of **ducks** that were present in the pond at the same time. This should look at the EntranceEvent and ExitEvent record types to construct a timeline and figure out what the peak occupancy is.

Remember that the list of names in BoundaryEvent records includes cow names. You'll need to exclude the cows from your count (*Hint: look at* `DaleRecord.java` *to see how you can measure cows*).

You can assume that the pond will always start with zero animals, and Exit events will not occur without a preceding Entrance event (i.e. the pond never has fewer than zero animals).

Throw an `IllegalArgumentException` if the pond name is null, and a `NoSuchElementException` if the pond is not in the database. This method must use only $O(log(r))$ auxiliary space, so you cannot store data for every record in a list or map, for example. The time complexity must be $O(r)$.

Hint: You may find `getClass()` or `instanceof` useful for this method.



Example 1: `getPeakConcurrentOccupancy("X")` returns **6**. This occurs at time 5, where there have been 8 entrances and 2 exits, so the total number of ducks in the pond is 6. Note: this assumes none of these entrance events included cows!

Example 2: `getPeakConcurrentOccupancy("Y")` returns **3**. This occurs at both time 3 and 5. Note: this assumes none of these entrance events included cows!

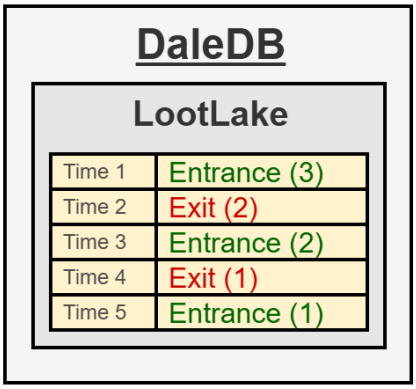
9. getMostFrequentVisitor

This method accepts a pond name, and returns the name of the most frequent visitor to this pond in $O(kr)$ time, where k is the number of names in the record with the most names. Either ducks or cows can be returned by this method; you need not differentiate the two.

There can be multiple animals who are the most frequent visitor. The method only needs to return one of them.

Throw an `IllegalArgumentException` if the pond name is null, and a `NoSuchElementException` if the pond is not in the database.

Example: Say we have the following records in DaleDB:




```
BoundaryEvent("LootLake", 1, ENTRANCE, ["Andy","Mike","Ham"], 0)
BoundaryEvent("LootLake", 2, EXIT, ["Andy","Ham"], 0)
BoundaryEvent("LootLake", 3, ENTRANCE, ["Andy","Ham"], 0)
BoundaryEvent("LootLake", 4, EXIT, ["Andy"], 0)
BoundaryEvent("LootLake", 5, ENTRANCE, ["Andy"], 0)
```

`getMostFrequentVisitor("LootLake")` returns **Andy**, who visited 3 times.

10. mergeReports

The Registry is willing to sacrifice some granularity in its records in order to speed up the lookup time for FishReports.

This method accepts a pond name and merges all FishReports from the same duck into a singular record. Records for each duck are consolidated by combining all fish weight data into one singular list in the record with the latest timestamp, and deleting all records before it.

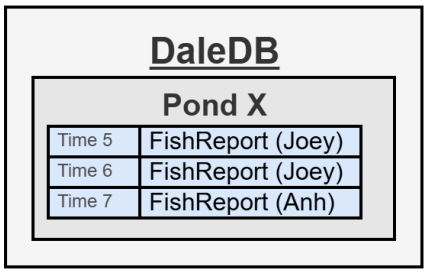
In addition to the merging, this method returns a map, where each key is the latest timestamp for each duck (the timestamp of the one remaining record), and each value is a list of timestamps that were merged into this record.

Throw an `IllegalArgumentException` if the pond name is null, and a `NoSuchElementException` if the pond is not in the database.

The time complexity for this method must be $O(r + klog(r))$, where k is the number of FishReports, and $k << r$.

Hint: Timestamps are unique for ponds, so two ducks cannot have the same FishReport timestamp within a pond. Also, records are immutable, so you cannot update a DaleRecord directly – you can only delete and replace it.

Example: Let's say we have the following records in DaleDB:




```
FishReport("X", 5, "Joey", [0.1, 0.2, 0.3])
FishReport("X", 6, "Joey", [0.3, 0.2, 0.1])
FishReport("X", 7, "Anh", [0.1, 0.2, 0.3])
```

`mergeReports("X")` will condense the records in Pond X to the following:

```
FishReport("X", 6, "Joey", [0.1, 0.2, 0.3, 0.3, 0.2, 0.1])
FishReport("X", 7, "Anh", [0.1, 0.2, 0.3])
```

And return the following map:

```
{6: [5], 7: []}
```

Joey has two records, with timestamps 5 and 6, so 5 gets merged into 6 and is removed from the pond. Anh only has one record, which remains unchanged.

Extension: The Big Picture

Note: this section is optional reading, and explains the real-world relevance of this project. Understanding this section is not required to complete the project; however, the language may be useful if you list this project on a professional document (e.g. resume). However, you may not post any of your code publicly.

DaleDB loosely models the idea of a NoSQL Database. NoSQL is a flavor of non-relational database that often holds information in the form of documents and/or key-value pairs. Conceptually, you can often visualize it like a big JSON object with keys that map to complex objects that constitute the actual data. A popular example of a NoSQL database is Amazon DynamoDB.

NoSQL databases like this often partition (split) the database by a hashable key, to allow for constant time access to that shard (segment) of data. Think about how you split the database based on a pond string in DaleDB – it's the same idea. Within a shard, there is often an auxillary key (called a sort key) to further organize multiple objects within the shard and optimize for object lookup – think about how you used timestamps to organize DaleRecords within a pond.

Since NoSQL databases are not like a table in a relational database, they can store hetereogenous data (multiple data types/schema in the same structure), and can support nested types (like how we can store multiple different types of DaleRecords in DaleDB, and how some of the attributes within these types were themselves data structures like lists). Note that Java's strict typing means that DaleDB does not fully utilize a schema-less structure, which is typical in NoSQL databases.

Having efficient access to complex data in partitions is a common configuration in software development, so a NoSQL database is a frequently used tool that is incorporated into many infrastructures. Understanding how they work could be very useful, and is sure to impress your friends, family, and interviewers!

Person

name	phone
John	3634
Sue	6343

Orders

personName	date	product
John	2002	Gizmo
John	2004	Gadget
Sue	2002	Gadget

```
{“Person”:
 [{"name": "John",
   "phone":3646,
   "Orders":[
    {"date":2002,"product":"Gizmo"},
    {“date”:2004,"product":"Gadget"}
   ]
  },
  {"name": "Sue",
   "phone":6343,
   "Orders":[
   {"date":2002,"product":"Gadget"}
   ]
  }
 ]
}
```

Image credit: Database Systems at UC Berkeley

The left side shows how an SQL database would be represented with relational tables, and the right side shows how NoSQL data is organized (in this example, with JSON).

Grading

Here is the grading breakdown for this project. There are various deductions not listed that are incurred when breaking the rules listed in this document, including inefficient code.

Part	Component	
1	HashMap	10%
2	TreeMap	15%
3	DaleDB	25%
All	Demo	50%

Autograder

The autograder for this course will give you some helpful information, but it will be limited. Every time you submit, you will receive info about any compiler errors, Checkstyle deductions, and general formatting issues. The **first three times per day** you submit, you will see your per-method score. You will not see the number of failed tests or any information about the tests themselves.

Note that while this score is generally representative, **it is not final**. We go over each submission manually, and it is possible for autograder points earned to be deducted in certain cases, including, but not limited to, attempting to brute-force test cases. Additionally, we will manually grade efficiency, so that score is not included in the autograder score. Finally, **we reserve the right to update our test cases** should any defects be discovered, which may result in slight changes to your autograder score.

Checkstyle

Your code must comply with all of the Checkstyle requirements. You will lose a half-point for each Checkstyle issue, capped at -5.0 points. Gradescope will tell you if there are Checkstyle issues in your submission, and you can also manually run Checkstyle by downloading it from the Course Website.

Collaboration Policy

Every student is expected to read, understand, and abide by the Georgia Tech Academic Honor Code.

When working on assignments, you **may not** directly copy code from any source (other than your own past submissions). You are welcome to collaborate with peers and consult external resources, but you **must** personally write all of the code you submit. **You must list, in your Main.java file, every student with whom you collaborated and every resource you consulted while completing the assignment**.

You may not directly share any files containing assignment code with other students or post your code publicly online. If you wish to store your code online in a personal private repository, you can use GitHub Enterprise to do this for free.